How to Improve the Substitutability Judge

A plain-language plan for raising the score, based on two analyses: (1) which of our programs are weak and why, and (2) what a router can and cannot add.

Where we are today

- Our programmatic judge reproduces the **LLM's** substitutability decisions about **75%** of the time (held-out, 721 cases).
- It agrees with **human** labels about **80%** of the time (35 clean cases).
- The LLM we are copying agrees with humans about **91%** of the time.
- **Target:** agree with the LLM **95-99%** of the time, while sending as few cases as possible to the (expensive) LLM.

Two independent levers close this gap: **(1) make the programs better** and **(2) add a router that sends only the hard cases to the LLM.** They work best together.

Part 1 - Fixing the weak programs

What we found. Of 80 programs, about 15 add little or no signal. They cluster in a few "rubrics": *specificity, modifier/qualifier, processing/additive, food-category*. The strong ones are the direct text-matchers: *same-food, preparation-state, preservation, word-overlap*.

Why they fail (in plain terms):

1. **They assume the two foods already match** and only look for *conflicting* words. When there is no conflicting word, they return a high score for almost everything, so they can't tell a good match from a bad one - e.g. "chia seeds" vs "flaxseed" slips through as "substitutable." *Fix: first check the core food actually matches, then apply penalties.*
2. **They match single words out of context** - like "with" or "in" - which appear in many entries, triggering penalties almost at random. *Fix: match whole phrases like "with oil" or "in brine," not loose single words.*
3. **Some programs rank cases correctly but output on a squashed scale**, so the single yes/no cutoff we use mislabels them and their good information is thrown away. *Fix: keep their raw scores and let the combiner use them, instead of forcing an early yes/no.*

Also important: a few programs look "bad" only because they disagree with the LLM - but they actually agree with **humans**. Since the LLM labels themselves are only 91% right versus humans, some "bad" programs may in fact be good. We should not over-tune to the LLM's answers.

Quick wins (no new LLM calls): - Let a program **abstain ("no opinion")** when it detects nothing relevant, instead of returning a default high score. This removes noise from the committee. - **Drop the ~3 programs that are no better than a coin flip.** - Let the combiner use **continuous scores**, not a forced yes/no, so well-ranked programs still count. - **Repair the over-strict programs** that reject harmless differences ("diced," "thawed," "raw") which neither humans nor the LLM treat as a problem.

Expected effect: a few points of higher base accuracy, and - just as important - a cleaner confidence signal, which makes the router (Part 2) cheaper.

Part 2 - Adding a router

What it is. After the programs vote, measure how confident the committee is. Decide the **confident** cases locally for free; send only the **unsure** cases to the LLM. Any case sent to the LLM gets the LLM's own answer, so it agrees with the LLM by definition.

Why it helps the most. Our committee is ~75% right, so ~25% are wrong. If we could send *exactly* those wrong 25% to the LLM, we'd reach ~100% agreement while paying for only a quarter of cases. In practice we can't spot the wrong ones perfectly:

- **Best possible ("oracle"):** send ~**20-24%** to the LLM -> **95-99%** agreement.
- **Realistic (confidence-based):** needs to send somewhat more, because confidence isn't perfect - but still a minority of cases.

Its limits: - It **does not fix the programs** - it routes around them. It **cannot catch cases where many programs are confidently wrong together** (those look "sure"), which is exactly why Part 1 matters. - **Ceiling:** cases sent to the LLM get the LLM's answer, and the LLM is only ~91% right vs humans. So measured against *human* truth, routing can't exceed ~91%. Against the LLM's own labels it can approach 100%.

Why do both

- **Part 1** raises base accuracy and makes the confidence signal trustworthy -> the router then reaches 95% by sending **fewer** cases to the LLM (lower cost).
- **Part 2** turns whatever errors remain into a controlled, measurable LLM spend, with a guarantee like *"send X% to the LLM, hold accuracy above Y%."*

In one line: the router is what actually gets us to 95-99%; fixing the weak programs is what makes reaching it **cheap**.

Recommended order of work

1. **De-noise the committee** - abstain-when-silent, drop coin-flip programs, use continuous scores. Cheap, no new LLM calls.
2. **Repair the over-strict programs** on the specific cases they get wrong.
3. **Build the router** and plot accuracy vs. "% sent to the LLM" (both the best-possible and the realistic version), *before and after* step 1 - to show how much cheaper good programs make routing.
4. **Grow the human-labeled set** (currently 35 clean cases) so we measure true quality, not just agreement with an imperfect LLM.